LLM-native semantic layer: LangFuse is the standout open source platform for LLM observability — providing prompt versioning, session tracing, token cost attribution, eval score capture, and a replay interface for debugging production failures. OpenLLMetry bridges the two worlds: it auto-instruments LangChain, OpenAI, Anthropic, and others — emitting OTel-compatible spans so LLM traces flow into your existing Jaeger or Tempo backend. Arize Phoenix, Evidently AI, and MLflow address evaluation and drift detection — closing the feedback loop between production telemetry and model quality assessment.

Reference architecture: AI observability stack

The architecture in Figure 1 shows a production-grade, open source AI observability stack structured across four logical tiers — provider-agnostic, pipeline-aware, and feedback-connected. It treats observability as a first-class architectural concern, not a bolt-on monitoring afterthought.

Tier 1 — Instrumentation: Every AI component is instrumented at source: LLM API calls, vector database queries, retrieval steps, tool invocations, and orchestration decisions. OpenLLMetry provides auto-instrumentation for major LLM frameworks. Custom spans capture domain-specific events such as RAG relevance scores and agent decision branches. Helicone or a custom OTel-enabled reverse proxy captures API-level telemetry without code changes in legacy services.

Tier 2 — Collection and transport: The OTel Collector aggregates telemetry from all sources, applies sampling strategies, enriches spans with environment metadata (model version, prompt version, deployment region, cost attribution), and routes to backend stores. This tier decouples instrumentation from storage — a critical design choice that lets you swap backends without touching application code.

Tier 3 — Storage and analysis backends: Traces route to Tempo or Jaeger. Metrics go to Prometheus. Logs land in Loki. LLM-specific semantic data — prompt/response pairs, eval scores, session trees — is stored in LangFuse's PostgreSQL-backed store. This separation of concerns is intentional: infrastructure telemetry and LLM semantic telemetry have different retention, query, and governance requirements.

Tier 4 — Visualisation, alerting, and feedback: Grafana provides the unified observability interface — dashboards spanning infrastructure metrics, LLM latency, token costs, and quality scores. Alertmanager enforces SLOs. Arize Phoenix and Evidently AI power the evaluation feedback loop — taking production traces, running automated quality scoring and drift detection, and routing failures back to prompt engineering or fine-tuning workflows. This tier is where observability becomes a continuous improvement engine.

Best practices for enterprise AI observability

Getting the tooling in place is the easy part. Operating it well — at enterprise scale, with governance requirements, across multiple AI products — requires deliberate architectural decisions:

Best practice	Why it matters
Instrument at pipeline level, not just model level	LLM calls are one node in a chain — capture the full span tree: retrieval, pre/post-processing, routing
Set baseline SLOs before production	Define p95 latency, token-per-second throughput, and hallucination rate thresholds before go-live — not after incidents
Separate observability by persona	Ops teams need infra metrics (GPU, queue depth); product teams need behavioural metrics (prompt success rate, session quality)
Capture prompts and outputs selectively	Log full prompt/response pairs for debugging — apply sampling and PII scrubbing. Full capture at scale is cost-prohibitive and risky
Track cost as a first-class metric	Token consumption maps directly to cost — alert on token spikes, model drift, and unexpected routing to expensive models
Use semantic versioning for prompts	Treat prompts as deployable artifacts. Correlate observability data to specific prompt versions to isolate regressions
Build feedback loops from observability to eval	Failing traces should feed automated eval pipelines — close the loop between prod telemetry and model quality
Plan for multi-model and multi-provider visibility	Abstract instrumentation at the gateway layer (Helicone, OpenLLMetry) for unified visibility across all providers

Top use cases: Where observability changes outcomes

Observability is not an IT concern — it is a business risk management capability. The following use cases illustrate where the absence of AI observability creates material business risks, and how open source tooling addresses it.